Projet MicroK8s
AI Jobs Market Dashboard

Une application web sur deux clusters MicroK8s

Simon Almeida
ESIEA Paris - 4A
Cours Virtualisation / Kubernetes
2026-2027

Table des matières

1. Introduction..... 3
2. Le dataset..... 3
3. Architecture..... 3
4. Mise en place des VM et de MicroK8s..... 4
5. L'application..... 5
6. Image Docker et Kubernetes..... 6
    6.1 Image..... 6
    6.2 Objets Kubernetes..... 6
7. Tests..... 7
    7.1 API et auto-réparation d'un Pod..... 8
    7.2 Perte complète de cluster-a (T4 et T5)..... 8
    7.3 StatefulSet..... 11
8. Difficultés rencontrées..... 11
    8.1 Pods en erreur au premier déploiement..... 11
    8.2 Cluster B injoignable..... 11
    8.3 Redémarrage lent de cluster-a..... 12
    8.4 Problèmes avec les VM..... 12
9. Limites et améliorations..... 12
10. Conclusion..... 12

1. Introduction

Le but du projet est de prendre le fichier CSV « AI Jobs Market 2025-2026 », d'en faire une petite application web, de la mettre dans une image Docker, puis de la faire tourner sur Kubernetes avec MicroK8s, dans des VM VirtualBox. L'application doit continuer à répondre même si une partie de l'infrastructure tombe.

Le sujet prévoit 5 VM : deux clusters de deux nœuds et une VM HAProxy. Mon PC n'a que 16 Go de RAM, et les VM viennent d'une image Ubuntu avec bureau qui consomme beaucoup. J'ai donc fait une version réduite :

- 2 VM, une par cluster. Chaque cluster a un seul nœud.
- Pas de HAProxy : on accède à chaque cluster directement par son IP.

Le reste est fait : application avec KPI, filtres, graphiques et API, image Docker, Deployment avec probes, Service, Ingress, ConfigMap, Secret, StatefulSet, DaemonSet, Job, CronJob, et des tests de panne.

2. Le dataset

Le fichier contient 1 500 offres et 25 colonnes. Les colonnes les plus utiles sont la catégorie du poste, le pays, le niveau d'expérience, le salaire annuel en dollars, les compétences demandées (séparées par « | »), le score de demande et deux colonnes 0/1 : télétravail possible et poste lié aux LLM.

Avant de passer à Kubernetes, j'ai vérifié les chiffres avec pandas :

Indicateur	Valeur	Indicateur	Valeur
Offres	1 500	Demand score moyen	87,52
Salaire moyen	194 892 $	Offres remote-friendly	75,4 %
Salaire médian	180 000 $	Rôles LLM	21,8 %

Ce qu'on apprend : la catégorie la mieux payée est Architecture (251 577 $ en moyenne). Les USA ont un tiers des offres (515). La compétence la plus demandée est Python. Les postes LLM sont payés environ 8 % de plus que les autres (207 746 $ contre 191 309 $).

3. Architecture

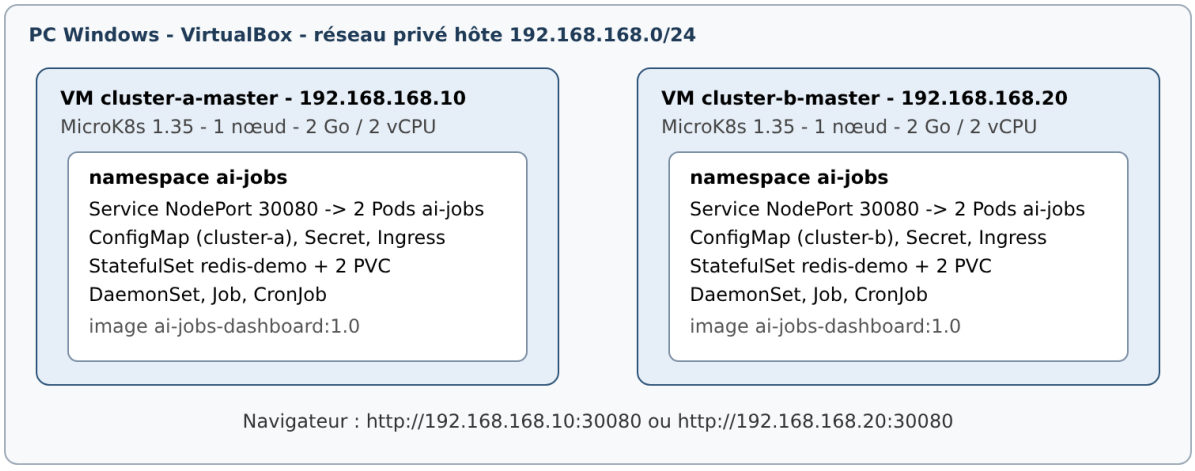


Figure 1 - Architecture : deux clusters indépendants, sans load balancer

Les deux clusters ne sont pas reliés entre eux. Chacun a son control plane, son stockage et sa copie de l'image. La même application tourne des deux côtés, seule la ConfigMap change (`CLUSTER_NAME`). Ça permet de voir sur la page quel cluster a répondu.

Chaque VM a deux cartes réseau : la carte 1 sur le réseau privé hôte avec une IP fixe, pour que les VM et Windows se parlent, et la carte 2 en NAT pour avoir Internet (snap, apt, Docker Hub). J'ai gardé la plage 192.168.168.0/24 qui existait déjà sur mon PC, au lieu du 192.168.56.0/24 du sujet.

VM	Rôle	IP	RAM / CPU
cluster-a-master	Cluster A (1 nœud)	192.168.168.10	2 Go / 2
cluster-b-master	Cluster B (1 nœud)	192.168.168.20	2 Go / 2

4. Mise en place des VM et de MicroK8s

J'ai cloné une VM Ubuntu 24.04 que j'avais déjà, en clones liés (ils partagent le disque de base, ça prend peu de place) avec de nouvelles adresses MAC. Ensuite, sur chaque VM :

```
sudo hostnamectl set-hostname cluster-a-master
sudo nmcli con add type ethernet ifname enp0s3 con-name hostonly ipv4.method manual \
  ipv4.addresses 192.168.168.10/24 ipv4.never-default yes ipv6.method disabled
sudo nmcli con up hostonly
sudo snap install microk8s --classic --channel=1.35
sudo microk8s enable dns
sudo microk8s enable hostpath-storage
sudo microk8s enable metrics-server
sudo microk8s enable ingress
```

J'ai utilisé NetworkManager (nmcli) et pas netplan, parce que l'image Ubuntu a un bureau. L'option `never-default` laisse Internet passer par la carte NAT. Les addons servent à : dns pour que les Pods trouvent les Services par leur nom, hostpath-storage pour les volumes du StatefulSet, metrics-server pour `kubectl top`, ingress pour l'Ingress (Traefik).



Figure 2 - Les VM dans VirtualBox (seules cluster-a-master et cluster-b-master servent)

```
NAME              STATUS   ROLES    AGE    VERSION   INTERNAL-IP      EXTERNAL-IP   OS-IMAGE           KERNEL-VERSION
   CONTAINER-RUNTIME
cluster-a-master   Ready    <none>   116s   v1.35.6   192.168.168.10   <none>        Ubuntu 24.04 LTS   6.8.0-31-generic
   containerd://2.1.6
lo                UNKNOWN        127.0.0.1/8 ::1/128
enp0s3            UP             192.168.168.10/24
enp0s8            UP             10.0.3.15/24 fd17:625c:f037:3:d2c8:1d0c:9aeb:ef6/64 fd17:625c:f037:3:ef4:7704:b0c1:33f3/
64 fe80::7589:b858:cc48:6ede/64
docker0           DOWN           172.17.0.1/16
vxlan.calico      UNKNOWN        10.1.57.192/32 fe80::64cd:d9ff:fe1c:3c25/64
cali76708f33efc@if2 UP              fe80::ecee:eeff:feee:eeee/64
cali148445b1737@if2 UP              fe80::ecee:eeff:feee:eeee/64
calif40cced40fb@if2 UP              fe80::ecee:eeff:feee:eeee/64
cali69d304149e6@if2 UP              fe80::ecee:eeff:feee:eeee/64
```

Figure 3 - Nœud cluster-a-master Ready, avec l'IP 192.168.168.10

5. L'application

L'application est en Python avec Flask et pandas. J'ai séparé les calculs (`analytics.py`) et le serveur web (`app.py`). Le CSV est lu une seule fois au démarrage.

Route	Ce qu'elle fait
/	Le dashboard : 7 KPI, filtres pays et catégorie, 5 graphiques, tableau des offres, bloc Cluster / Pod / Node
/health	Répond {"status":"ok"}. Utilisée par les probes Kubernetes.
/api/stats	Les KPI en JSON, avec les filtres ?country= et ?category=
/api/jobs	Les offres filtrées en JSON

Le nom du cluster vient de la ConfigMap. Le nom du Pod et du nœud sont donnés par Kubernetes avec `fieldRef`. Chaque requête est écrite dans les logs avec la date, la route, le cluster, le Pod, le code HTTP et la durée. J'ai aussi écrit 4 tests pytest (health, KPI globaux, filtre France, page d'accueil).

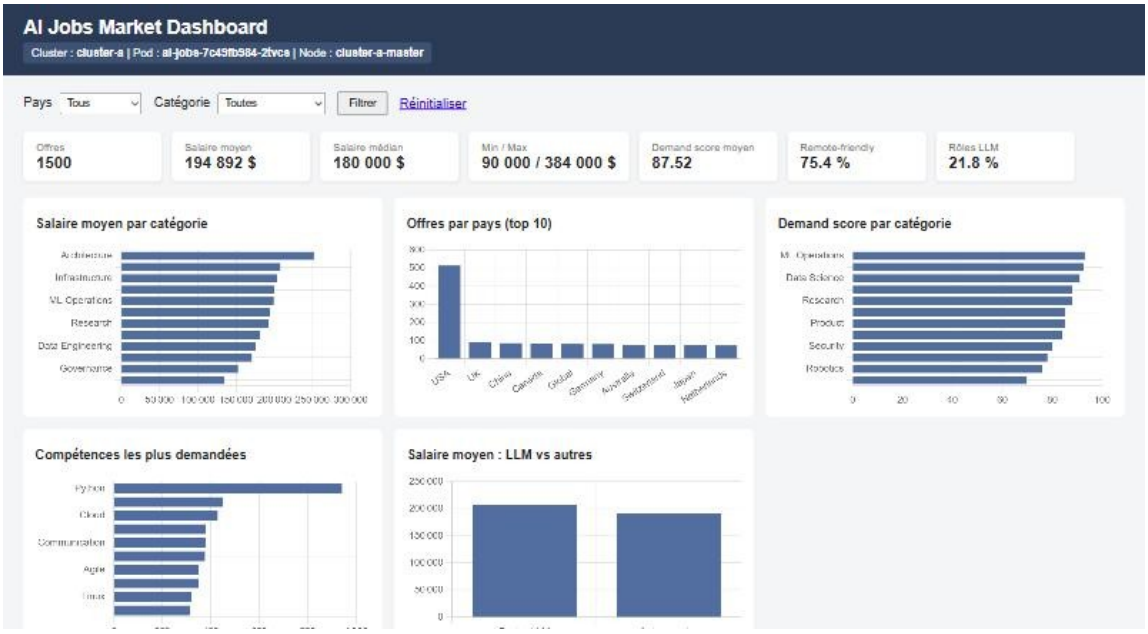


Figure 4 - Dashboard servi par cluster-a (Pod ai-jobs-...-2tvcs, nœud cluster-a-master)

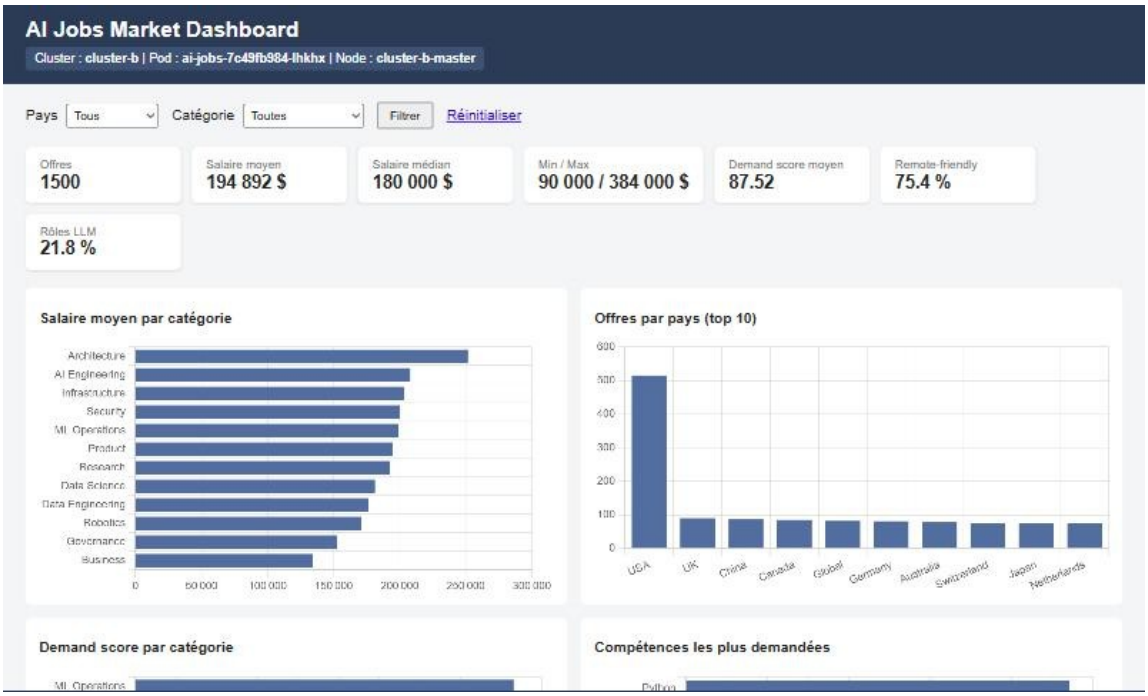


Figure 5 - Le même dashboard servi par cluster-b (Pod ai-jobs-...-lhkhx, nœud cluster-b-master)

6. Image Docker et Kubernetes

6.1 Image

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ /app/
COPY data/ /app/data/
RUN chmod -R a+rX /app && useradd -m appuser
USER appuser
EXPOSE 8000
CMD ["gunicorn", "--workers", "2", "--bind", "0.0.0.0:8000", "app:app"]
```

L'application tourne avec un utilisateur normal (pas root) et Gunicorn. Je n'ai pas de registre, donc j'ai pris l'option « import local » du sujet : le script `deploy.sh` construit l'image avec Docker sur la VM, puis l'importe dans MicroK8s avec `microk8s ctr image import`. Il faut le faire sur chaque cluster, car chaque nœud doit avoir l'image en local. C'est pour ça que le Deployment est en `imagePullPolicy: IfNotPresent`.

6.2 Objets Kubernetes

Objet	Réglages
Deployment ai-jobs	2 replicas, mise à jour progressive (maxUnavailable 0, maxSurge 1), anti-affinity preferred, requests 50m / 128Mi, limits 300m / 384Mi, readiness et liveness sur /health
Service	NodePort 30080 → port 80 → port 8000 du conteneur
Ingress	ai-jobs-a.local et ai-jobs-b.local
ConfigMap / Secret	CLUSTER_NAME et APP_TITLE / un token d'exemple
StatefulSet redis-demo	2 replicas, un volume de 256Mi par Pod

Objet	Réglages
DaemonSet node-agent-demo	Un Pod busybox par nœud qui écrit le nom du nœud
Job / CronJob	csv-check (une fois) / health-report (appelle /health toutes les 5 min)

J'ai mis 128Mi de request mémoire au lieu de 64Mi : avec le CSV chargé dans pandas, un Pod prend environ 108Mi (mesuré avec `kubectl top`). L'anti-affinity est en « preferred » : avec un seul nœud, une règle « required » laisserait le deuxième Pod en Pending.

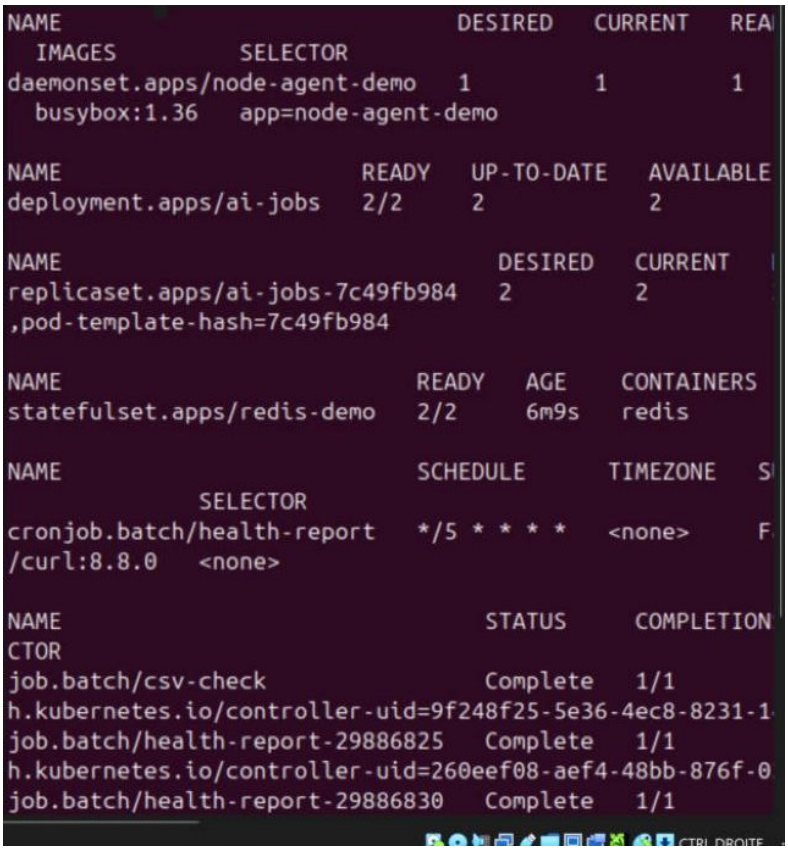


Figure 6 - Après le déploiement sur cluster-a : Deployment 2/2, StatefulSet 2/2, DaemonSet, CronJob et Jobs terminés

7. Tests

Test	Ce que j'ai fait	Résultat
T1	Vérifier les nœuds des 2 clusters	OK, les deux Ready
T2	Supprimer un Pod ai-jobs	OK, recréé tout seul
T3	Arrêter un worker	Pas faisable, pas de worker séparé
T4	Éteindre tout le cluster A	OK, B répond pendant toute la panne
T5	Rallumer le cluster A	OK, A revient tout seul
API	/health, /api/stats et filtres	OK sur les 2 clusters
StatefulSet	Supprimer redis-demo-0	OK, même nom et même volume
DaemonSet / métriques	Logs de l'agent, kubectl top	OK

7.1 API et auto-réparation d'un Pod

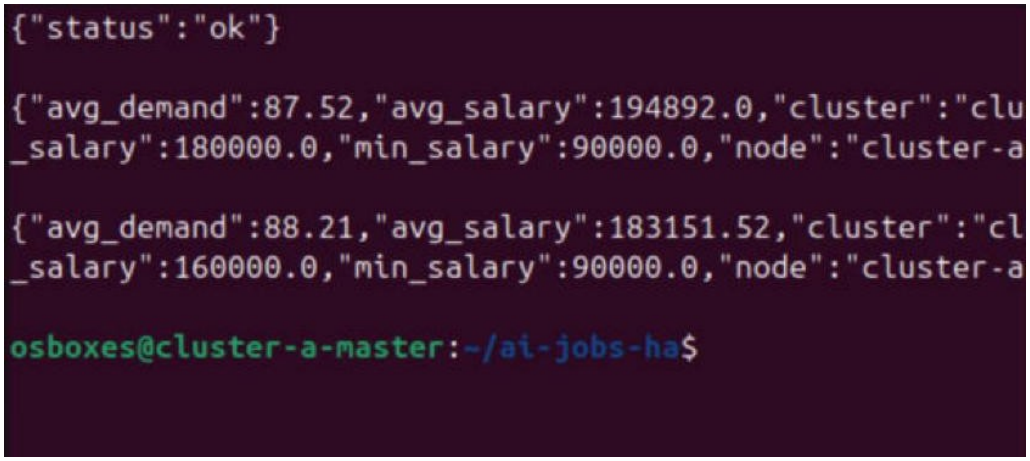


Figure 7 - API de cluster-a : /health, puis /api/stats sans filtre (1 500 offres) et avec le filtre France (66 offres)

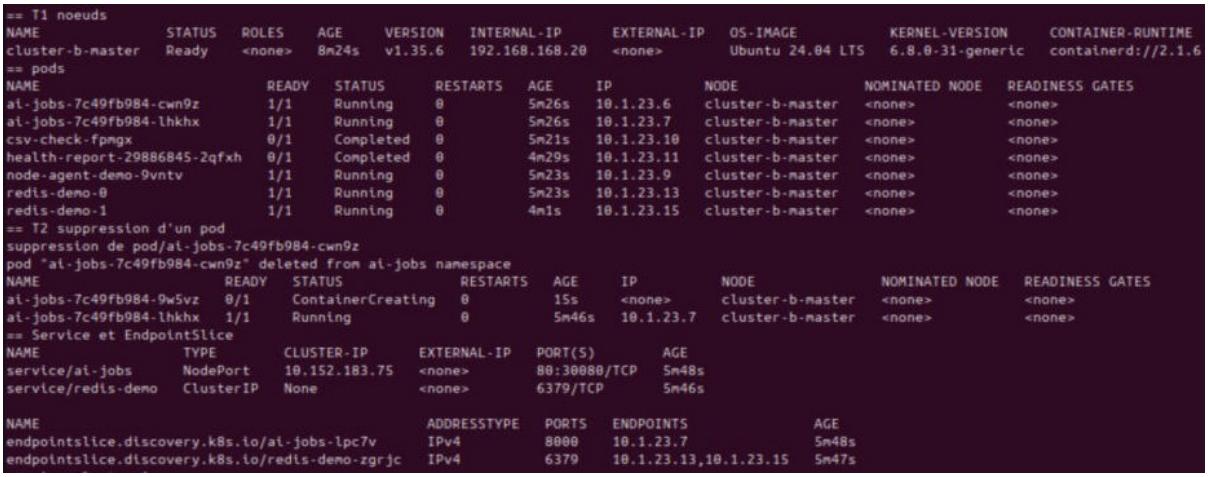


Figure 8 - Sur cluster-b : je supprime le Pod ...-cwn9z, et Kubernetes crée tout de suite ...-9w5vz

Ce n'est pas le Pod supprimé qui revient : le ReplicaSet voit qu'il n'y a plus qu'un Pod au lieu de deux, et il en crée un nouveau. Pendant ce temps, l'EndpointSlice ne contient que le Pod prêt, donc le Service n'envoie pas de trafic au nouveau Pod tant que sa readiness probe ne répond pas.

7.2 Perte complète de cluster-a (T4 et T5)

C'est le test le plus important. Sur la VM B, une petite boucle appelle `/health` des deux clusters toutes les 2 secondes et affiche le code HTTP (200 = OK, 000 = injoignable). Ensuite, je coupe la VM A dans VirtualBox avec « Éteindre », ce qui revient à débrancher la machine.

```
while true; do
  T=$(date +%T)
  A=$(curl -s -m 2 -o /dev/null -w "%{http_code}" http://192.168.168.10:30080/health)
  B=$(curl -s -m 2 -o /dev/null -w "%{http_code}" http://192.168.168.20:30080/health)
  echo "$T   cluster-a: $A   |   cluster-b: $B"
  sleep 2
done
```



Figure 9 - cluster-a-master éteinte, cluster-b-master toujours en fonction

```
=== TEST HAUTE DISPONIBILITE - surveillance des 2 clusters (200 = OK, 000 = injoignable) ===
17:21:39   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:41   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:43   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:45   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:47   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:49   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:51   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:53   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:55   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:21:57   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:00   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:02   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:04   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:06   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:08   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:10   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:12   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:14   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:16   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:18   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:20   cluster-a (192.168.168.10): 200 OK     |   cluster-b (192.168.168.20): 200 OK
17:22:23   cluster-a (192.168.168.10): 000 DOWN   |   cluster-b (192.168.168.20): 200 OK
17:22:27   cluster-a (192.168.168.10): 000 DOWN   |   cluster-b (192.168.168.20): 200 OK
17:22:31   cluster-a (192.168.168.10): 000 DOWN   |   cluster-b (192.168.168.20): 200 OK
17:22:35   cluster-a (192.168.168.10): 000 DOWN   |   cluster-b (192.168.168.20): 200 OK
```

Figure 10 - À 17:22:23, A passe à 000 DOWN. B reste à 200 OK.

```
=== PENDANT LA PANNE DE CLUSTER-A - 17:25:24 ===
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1008
pipe 2

NAME              STATUS   ROLES    AGE   VERSION
cluster-b-master   Ready    <none>   27h   v1.35.6
NAME                      READY   STATUS    RESTARTS   AGE   IP
GATES
ai-jobs-7c49fb984-9w5vz   1/1     Running   0          27h   10.1.23.17
ai-jobs-7c49fb984-lhkhx   1/1     Running   0          27h   10.1.23.7

"cluster":"cluster-b"
"jobs":1500
"node":"cluster-b-master"
"pod":"ai-jobs-7c49fb984-lhkhx"
osboxes@cluster-b-master:~$
```

Figure 11 - Pendant la panne : A ne répond plus au ping, B est Ready avec ses 2 Pods et répond à l'API

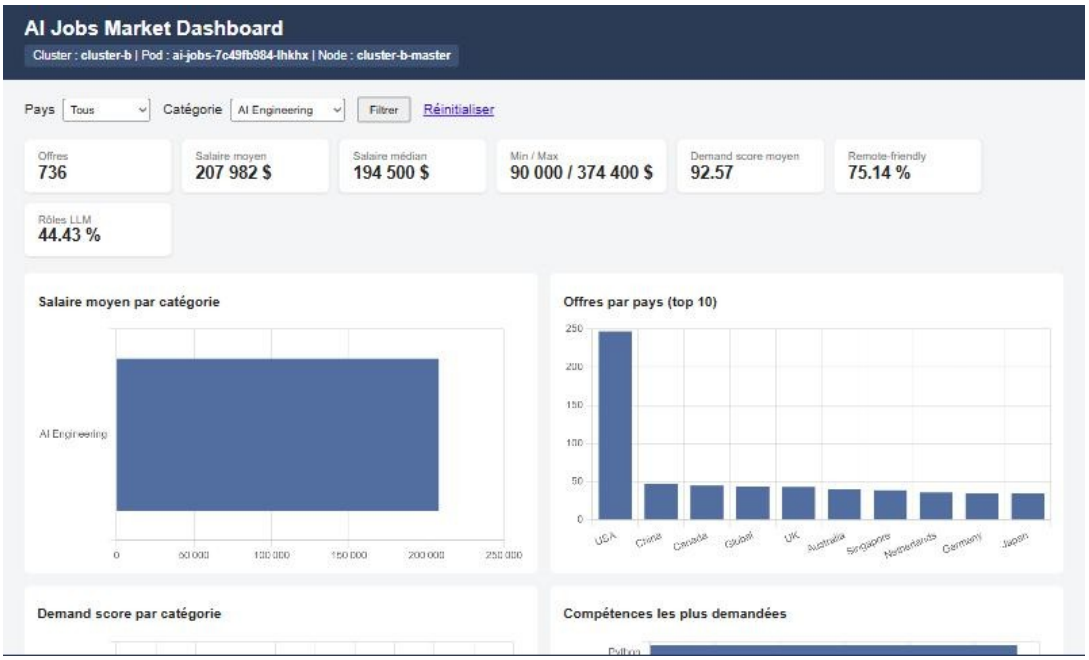


Figure 12 - Pendant la panne, le dashboard de cluster-b marche normalement, filtres compris

Ensuite j'ai rallumé la VM A, sans lancer aucune commande Kubernetes. MicroK8s redémarre et recrée tout ce qui est décrit dans les YAML.

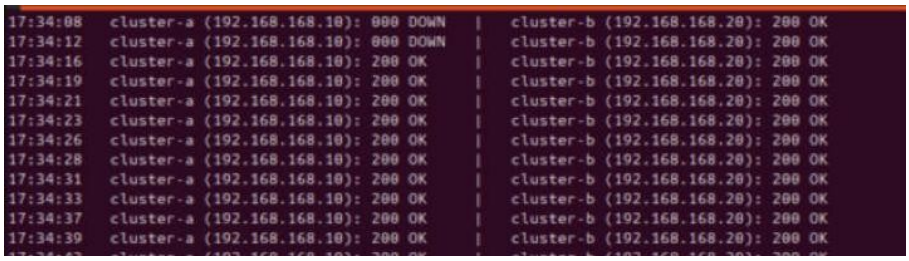


Figure 13 - A revient à 200 OK à 17:34:16

Heure	Événement	cluster-a	cluster-b
17:22:20	Dernière réponse de A	200	200
17:22:23	VM A coupée	000	200
vers 17:27	VM A rallumée	000	200
17:34:16	A répond de nouveau	200	200

Pendant les 12 minutes où A était en panne, B n'a pas raté une seule réponse. Le service est donc resté disponible. Par contre, sans load balancer, quelqu'un qui utilisait l'adresse de A doit passer lui-même sur celle de B. A a mis environ 7 minutes à revenir après le redémarrage (voir les difficultés).

7.3 StatefulSet

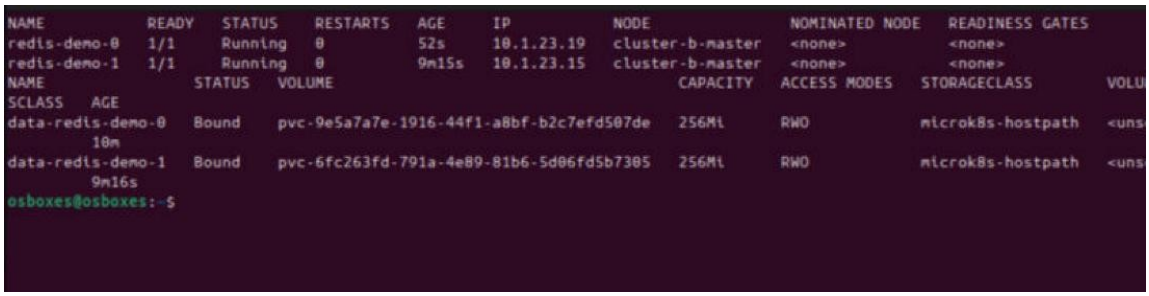


Figure 14 - Après suppression, redis-demo-0 revient avec le même nom et garde son volume data-redis-demo-0

C'est la différence avec un Deployment : un Pod de StatefulSet garde son nom et son volume, alors qu'un Pod de Deployment est recréé avec un nom au hasard.

8. Difficultés rencontrées

8.1 Pods en erreur au premier déploiement

Au premier déploiement, les deux Pods ai-jobs étaient en Error et redémarraient en boucle. J'ai regardé les logs du conteneur précédent avec `kubectl logs <pod> --previous`, qui montraient une `PermissionError` sur le CSV. Le fichier avait les droits 600, donc seul root pouvait le lire, alors que l'image tourne avec `appuser`. J'ai ajouté `chmod -R a+rX /app` dans le Dockerfile, reconstruit l'image et redéployé. Leçon : tester l'image avec `docker run` avant de la mettre dans Kubernetes, comme le dit le sujet.

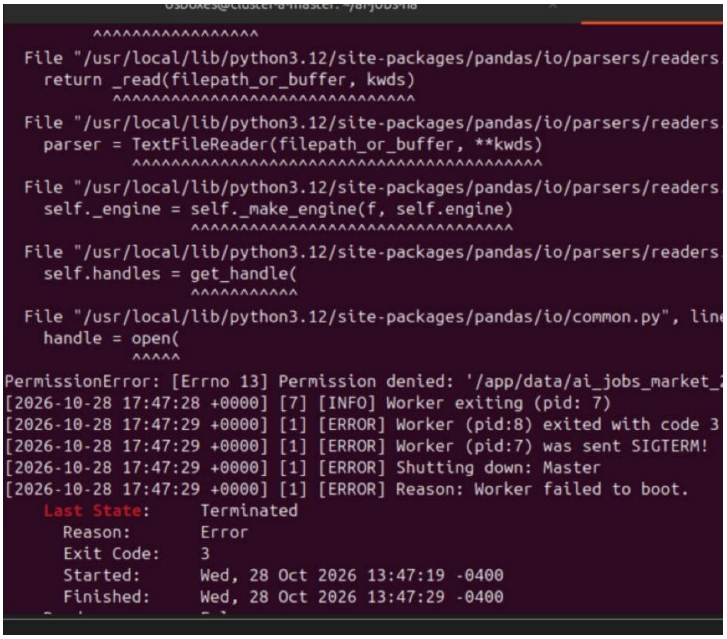


Figure 15 - L'erreur trouvée avec kubectl logs --previous

8.2 Cluster B injoignable

Depuis A, `ping 192.168.168.20` donnait « Destination Host Unreachable ». Depuis B, je n'arrivais même pas à joindre Windows (192.168.168.1), alors que l'IP était bien configurée dans la VM. Le problème venait donc de VirtualBox : la carte 1 de B était en NAT au lieu de « Réseau privé hôte ». Je l'ai changée pendant que la VM tournait (Périphériques > Réseau), et tout est revenu. J'ai compris qu'un problème réseau peut faire croire à une panne Kubernetes, et qu'il faut toujours tester la communication entre les VM.

8.3 Redémarrage lent de cluster-a

Après le test de panne, A a mis environ 7 minutes à revenir, et ses Pods affichaient 9 redémarrages. Au démarrage, la VM charge Ubuntu, MicroK8s et tous les Pods en même temps avec seulement 2 Go de RAM. L'application met alors plus de 15 secondes à démarrer, et la liveness probe la tue avant qu'elle soit prête. Il faudrait ajouter une `startupProbe` : tant qu'elle n'a pas réussi, la liveness ne s'applique pas. Je retiens qu'une probe mal réglée peut elle-même causer une panne.

8.4 Problèmes avec les VM

- Le copier-coller entre Windows et les VM ne marchait pas (Ubuntu 24.04 en Wayland). Il a fallu installer `virtualbox-guest-utils` et `virtualbox-guest-x11`, puis lancer `VBoxClient --clipboard`.
- Après `usermod -a -G microk8s`, il faut se reconnecter. Avant, `microk8s kubectl` sans sudo répond « Insufficient permissions ».
- Avec 2 Go par VM, tout est lent (construction de l'image, démarrage de MicroK8s), et la VM utilise le swap. C'est ce qui m'a fait passer de 5 VM à 2.

9. Limites et améliorations

- Un seul nœud par cluster : si la VM tombe, tout le cluster tombe. Avec un worker en plus, les deux Pods seraient sur deux nœuds.
- Pas de HAProxy : la bascule entre les clusters n'est pas automatique. Il faudrait une VM HAProxy qui teste `/health` sur les deux NodePort et envoie le trafic vers le cluster en bonne santé. HAProxy deviendrait alors à son tour un point unique de panne, qu'on peut supprimer avec deux HAProxy et Keepalived.
- hostpath-storage garde les données sur le disque du nœud : ce n'est pas un stockage haute disponibilité.
- Sécurité simplifiée pour le TP : Secret d'exemple, HTTP sans TLS, pas de NetworkPolicy.

10. Conclusion

L'application tourne sur deux clusters MicroK8s indépendants, et les tests montrent plusieurs niveaux de résilience. Un Pod supprimé est recréé par le Deployment. Un Pod de StatefulSet garde son nom et son volume. Quand un cluster entier tombe, l'autre continue de répondre.

Ce que je retiens : la haute disponibilité dépend du type de panne. Deux Pods protègent contre la perte d'un Pod, pas contre la perte de la VM. Deux clusters protègent contre la perte d'un cluster, mais il faut un load balancer pour que la bascule soit automatique. Les problèmes rencontrés m'ont aussi appris à regarder les logs et à tester le réseau avant de modifier la configuration.